ChatGPT

Executive Summary

This project ("EconoCausal") uses causal inference to measure how offering discounts (the "treatment") affects customer purchases (the "outcome") and then decides *which* customers should receive discounts under a fixed budget. The goal is to identify **who buys because of a discount** and how much extra revenue that generates, then allocate limited discount budget to maximize incremental sales. We will gather relevant retail/marketing data, preprocess it, and apply a Double Machine Learning (DML) model to estimate each customer's *Individual Treatment Effect* (ITE) of giving a discount. We then solve an optimization problem (a kind of knapsack or linear program) to allocate discounts within the budget. Key deliverables are the code pipeline (with reproducible notebooks or scripts), a final report (this document), a slide deck, and a demo or dashboard.

In practice, we must be realistic: we won't build an oracle system, but we will demonstrate the full pipeline on example data and synthetic data. The report below explains each step in simple terms, using examples and citing authoritative sources. We emphasize **causal assumptions, clear methodology, and rigorous validation** so our conclusions are defensible【25†L14-L21】【42†L43-L49】.

Project Overview

Goal: Measure the causal effect of discounts on purchases and optimize discount allocation under a budget. In other words, find which customers truly *need* a discount to buy (and how much incremental revenue that produces), then target those customers rather than giving discounts to everyone.

Scope:

- **Causal Estimation:** Use observational data to estimate the impact (uplift) of discounts on purchasing. We will assume a *partially linear causal model* (sales = baseline + θ·(discount indicator) + noise)【25†L83-L91】.
- **Double Machine Learning (DML):** Apply modern causal methods (DML) that combine flexible ML models for nuisance functions with a robust estimator for the treatment effect【25†L14-L21】【25†L105-L110】.
- **Optimization:** Given per-customer treatment effects, solve a budget-constrained optimization (similar to a knapsack problem) to decide which customers get discounts.
- **Deliverables:** Clean data tables and code; a report of findings (this document); presentation slides; a demo/ dashboard showing results (e.g., uplift curves or a small web app).

Expected Deliverables:

- A **README** and well-commented code (Jupyter notebooks or scripts) that reproduce the analysis.
- A **project report** (this document) detailing methods, assumptions, and findings.
- A **slide deck** summarizing approach and results for non-technical stakeholders.
- A **demo** (could be a mock-up dashboard or results notebook) showing the ITE estimates, uplift/Qini curves, and optimized discount plan.

Assumptions: We assume we have historical customer-level data including past purchases, discount offered, and relevant features (age, previous spend, product categories, date, etc.). We assume *no unobserved confounders* (i.e. any factor affecting both discount assignment and purchase is measured) 【25†L14-L21】. We further assume sufficient variability in who got discounts so the model can learn effects. The final allocation assumes discounts have a fixed cost per customer (or per unit), and budget is a hard cap.

Team Skills, Roles, and Timeline

Completing this project in 12 weeks with a 4-person team requires diverse skills and clear role division. Below are suggested roles (roles can overlap) and a week-by-week plan.

Role / Person	Key Skills	Responsibilities
Data Engineer	Data collection/ETL, SQL/Python, data cleaning.	Identify and gather datasets (public or internal). Clean and merge data, handle missing values, generate features (e.g. date parts, customer segments), and prepare the final data tables for modeling.
Causal Modeler	Statistics, machine learning, econometrics.	Design causal strategy (draw DAG, list confounders). Implement the DML model (choose ML learners, cross-fitting), estimate treatment effects. Run robustness checks (placebo, covariate balance, sensitivity).
Optimization Analyst	Operations research, Python (SciPy/pulp), math.	Formulate the budget-constrained optimization (objective and constraints). Code and solve the optimization (e.g. knapsack or ILP). Analyze resulting discount plan (ROI, incremental revenue).
Project Manager / Dev	Project planning, documentation, presentation.	Coordinate tasks and deadlines. Integrate components (data pipeline + model + optimization). Prepare the report and slides. Ensure code is version-controlled and reproducible (e.g. via Git).

Weekly Milestones (12-week plan):
We recommend two-week sprints, each producing a key milestone.

Week(s)	Milestones / Tasks
1–2	📖 **Literature & Data Survey:** Learn DML basics. Identify data sources (see next section). Sketch causal DAG and assumptions. Finalize project plan.
3–4	📥 **Data Acquisition & EDA:** Download public datasets or load sample data. Clean and preprocess (handle nulls, encode variables). Perform exploratory analysis.
5–6	🤖 **Model Prototype:** Implement first-stage models (predict purchase & discount assignment). Set up cross-fitting. Pilot DML to get initial ITE estimates.

Week(s)	Milestones / Tasks
7–8	🔍 **Validation:** Conduct robustness tests (placebo check, covariate balance, sensitivity). Tune models (hyperparameters) for better ITE estimates.
9–10	🧮 **Optimization & Integration:** Build the discount allocation optimizer (e.g. maximize total ITE * price). Integrate model + optimizer into one pipeline.
11	📊 **Evaluation & Visualization:** Compute evaluation metrics (e.g. Qini, uplift curves). Create recommended charts (e.g. ITE histogram, Qini curve). Finalize results.
12	📝 **Reporting & Demo:** Complete write-up (this report). Finalize slides. Polish demo/ dashboard. Perform final tests and rehearse presentation.

Each sprint should end with deliverables: e.g., a cleaned dataset, preliminary model results, charts, or a written summary.

Datasets (Public & Synthetic)

Option	Description & Source	Notes
Rossmann Store Sales 【42†L43-L49】	Historical daily sales from Rossmann drug stores (Kaggle competition). Includes whether a promotion ran each day (`Promo` flags)【42†L43-L49】【42†L123-L129】.	A multi-store retail dataset with `Sales`, `Customers`, `Open`, `Promo`, plus store info. (See [42]) Can use `Promo` as a proxy for discount.
Favorita Grocery Sales	**(Example)** Retail grocery data (Kaggle). Contains item-level sales and binary promotion flags (Promotion2).	(If accessible) Includes `unit_sales`, `onpromotion`. Commonly used for demand modeling. Can treat `onpromotion` as discount indicator.
Online Retail (UCI)	Retail transaction data (UK e-commerce). Contains `Quantity` and `UnitPrice` but *no explicit discount column*【42†L43-L49】.	Can infer discounts by tagging unusually low `UnitPrice` or synthetic coupons. Good for practicing data merging.
Synthetic Data (simulated)	Simulate customers/customers features X, treatment D = Bernoulli(discount_prob(X)), and outcome Y = f(X) + θ·D + noise.	E.g. use [19]'s `make_plr_CCDDHNR2018()` as a template【19†L58-L61】. Or manually generate: pick covariates (age, income), let `D ~ Bernoulli(sigmoid(wX))`, `Y = g(X) + θ*D + ε`.

Gathering Data: We will mainly use **open/public data**. For instance, the Rossmann Kaggle dataset (via an API or Kaggle download) is ideal because it explicitly includes promotion flags【42†L123-L129】. Alternatively, if data is scarce, we will **simulate** a plausible dataset: create customer-level features (demographics, past spend), randomly assign discounts based on some rule (e.g. high-spend customers are less likely to get discounted), and generate purchases with a known effect of the discount. This can be done

using Python (e.g., NumPy or DoubleML's built-in sampler【19†L58-L61】). Even if synthetic, we should code it to reflect a retail scenario (e.g., purchases increase when discounts are given).

Example Synthetic Setup (outline): Suppose 10,000 customers. For each customer *i*:

- Draw features X_i (e.g. age, location, prior spend).
- Treatment D_i (0/1) = 1 with probability $p(X_i)$ (a known function, e.g. logistic). This encodes selection bias if needed.
- Outcome Y_i = baseline $g(X_i) + \theta \cdot D_i + noise$. For example $Y_i = 100 + 5 \cdot age_i + \theta \cdot D_i + \varepsilon_i$. This creates a "ground truth" θ (the causal effect of discount) that our DML model should recover.

Data Collection & Preprocessing

- **Data Collection:** Obtain chosen dataset(s) via API or download. E.g. Rossmann data can be downloaded from Kaggle or an official source. Store raw data files (CSV).
- **Merge & Clean:** If using multiple files (e.g. customer file, transactions file), merge them on keys (e.g. customer ID). Remove rows with missing crucial fields (or impute if necessary). Drop irrelevant columns.
- **Feature Construction:** Create useful variables. Examples: convert `Date` into day-of-week, month, or holiday indicator; bucket continuous fields; encode categorical (One-Hot or ordinal). Ensure numerical features are scaled if using regularized models.
- **Treatment Column:** Define the treatment variable (e.g. `DiscountGiven = 1` if a discount was applied, else 0). If the dataset has a promotion flag (like Rossmann's `Promo`), use that. If not, create one synthetically (e.g. tag an order as "discounted" if the price is 5% below the typical price).
- **Outcome Column:** Usually sales or number of purchases. If modeling purchase probability (binary buy/not), ensure it's clearly defined. If modeling sales amount, note it can be skewed (may need log-transform)【25†L83-L91】.

Throughout, **Document** each step (especially any filtering or assumption e.g. "we drop customers with incomplete history"). We must not pretend the data was perfect – be honest about limitations (maybe some confounders are missing, which we'll flag in the causal assumptions section).

Feature Engineering

Useful features might include:

- **Customer history:** past purchase frequency or recency. A long-time loyal customer vs. a new customer might respond differently to discounts.
- **Demographics:** age group, income bracket, location. These could influence purchase propensity.
- **Product info:** category, price, competitor products. For example, if item price is already low, discounts may have less effect.
- **Time features:** seasonality (month, holiday season), day of week. People might naturally buy more around certain events, confounding discount effects.
- **Interaction features:** e.g. *age* × *discount*, or *income* × *discount*. We can feed these into models or let tree-based learners capture them.

We will standardize numeric features (z-score) and one-hot encode categoricals so our ML models work well. For example, if "Gender" is in data, turn it into binary. If "ProductCategory" has many levels, use target encoding or small embedding in advanced models (if needed).

Example: In Python, using pandas/sklearn:

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
encoder = OneHotEncoder(handle_unknown='ignore')
X_cat = encoder.fit_transform(df[['StoreType','PromoInterval']])
scaler = StandardScaler()
X_num = scaler.fit_transform(df[['CompetitionDistance','Month','Customers']])
```

(These are illustrative; we'll apply similar steps to our actual data.)

Causal Identification Strategy

We model sales `Y` as a function of giving a discount `D` and other factors `X`. A simple **causal diagram (DAG)** is:



This shows features X (e.g. past behavior, customer type) influence both *D* and *Y*. We **must account for all such confounders X**; otherwise our estimate of $D \to Y$ will be biased【25†L14-L21】. For example, suppose loyal customers (high spenders) are **less likely** to get discounts (so the discount is correlated with being a lower spender). If we ignored loyalty, a naive model might think "discounted customers buy *less*" simply because of that confounding. DML works by first fitting models for both `E[Y|X]` and `E[D|X]` and then orthogonalizing【25†L14-L21】【25†L105-L110】, which helps remove the bias from such confounders.

Key assumptions:

- **Unconfoundedness:** After conditioning on X (all observed features), the assignment of discount *D* is effectively random. In practice, this means we believe we have captured all major variables that influence offering a discount and also affect sales【25†L14-L21】. If, for instance, salespeople decided discounts based on an unrecorded metric (like a hunch or private info), that violates this assumption. We must be aware of such possibilities and mention them.
- **Positivity (Overlap):** For every customer profile in X, there is a nonzero chance of both receiving and not receiving a discount. Without overlap, we cannot compare treated vs control in some regions of X. If, say, very high-value customers *always* got discounts in our data, then we can't estimate the effect for them. We should check the distribution of D across X to ensure no "all-or-nothing" rules.

- **No interference:** Each customer's outcome is assumed unaffected by other customers' discounts (SUTVA). This is reasonable if customers decide independently.

Instrumental variables: We likely have none. Instruments would be variables that affect offering a discount but not sales directly (hard to find in such data). If an instrument existed (e.g. a randomized voucher campaign), we could use a DoubleMLIV model. In our analysis, we assume all influence of D on Y is through the discount itself, not via some hidden channel.

Double Machine Learning (DML) Implementation

Double ML (also called *debiased* ML) performs two modeling steps to estimate the causal effect θ of D on Y 【25†L14-L21】【25†L105-L110】:

1. **First-stage (nuisance) models:** Fit a model for `Y ~ X` and a model for `D ~ X` using any flexible ML (random forest, gradient boosting, etc.). This learns the "baseline" behavior of sales and discount assignment given features.
2. **Second-stage (orthogonalization):** Compute residuals `Ỹ = Y – Ŷ(X)` and `D̃ = D – D̂(X)`. Then regress `Ỹ` on `D̃` (often just via linear regression). The coefficient on `D̃` is the *causal effect* θ̂. The intuition is: by removing the part explained by X, we isolate the variation in D that is "as if random" and attribute its effect on Y【25†L105-L110】.

This method yields a *root-n consistent* and approximately unbiased estimate, even with complex X【25†L14-L21】【25†L105-L110】. It also gives valid confidence intervals. In Python, we can use libraries like EconML or DoubleML to implement DML easily. For example, a sketch with EconML:

```
from econml.dml import LinearDML
from sklearn.ensemble import RandomForestRegressor
# model_y: model for Y|X; model_t: model for D|X
est = LinearDML(model_y=RandomForestRegressor(max_depth=5),
                model_t=RandomForestRegressor(max_depth=5),
                discrete_treatment=True)
est.fit(Y=df['Sales'], T=df['Discount'], X=df[features])
ite = est.const_marginal_effect(df[features])  # array of ITE per customer
```

This code (based on [25]) fits the DML estimator and returns each customer's *constant marginal effect* (assuming a linear form). The actual doubleML library has similar syntax. By default it will do cross-fitting (splitting data folds) to avoid overfitting bias【56†L215-L223】.

Algorithms & Hyperparameters: We will likely use tree-based models for the nuisances (e.g. Random Forest, XGBoost, or LightGBM) because they handle heterogeneity and interactions well. We should tune key parameters (depth, number of trees, learning rate) via cross-validation or a library like Optuna. The `DoubleMLPLR` class in doubleML (Python) implements the above with various scoring options (partialling-out or IV-type)【56†L218-L226】. We might start with `score='partialling out'` (classical DML) and compare to `score='IV-type'` if needed.

Code Snippet (DoubleML example): For illustration, a simplified usage of DoubleML in Python might look like this:

```
from doubleml import DoubleMLData, DoubleMLPLR
from sklearn.ensemble import RandomForestRegressor

# Prepare DoubleML data object
dml_data = DoubleMLData(df, y_col='Sales', d_cols='Discount', x_cols=features)
# Initialize DML estimator with chosen ML models for nuisances
dml_plr = DoubleMLPLR(dml_data,
                      ml_l=RandomForestRegressor(max_depth=5),
                      ml_m=RandomForestRegressor(max_depth=5))
dml_plr.fit()  # fits the model
theta_hat = dml_plr.coef[0]    # estimated causal effect (scalar)
se = dml_plr.se[0]             # its standard error
```

This mirrors the official example in DoubleML's documentation【56†L218-L226】. The `coef` gives our estimate θ, and by calling `.fit()`, the library automatically does cross-fitting to correct for bias. We would extract the ITE for each customer from the model predictions (e.g. using the `.effect()` or similar methods provided).

Validation & Robustness Checks

Getting a number $\hat{\theta}$ is just the start; we must verify it makes sense.

- **Placebo Test:** Randomly shuffle the discount labels among customers and re-run the DML. If the model is sound, the estimated effect should be near zero for the placebo data. A large estimate would signal model or data issues.
- **Covariate Balance:** Check that key features are similarly distributed in the actual discount and no-discount groups *after weighting by the propensity scores* or similar. Tools like scikit-uplift or causal libraries have functions for this. If imbalances remain, our confounding adjustment may be insufficient.
- **Sensitivity Analysis:** Methods like the E-value or DoWhy's refutation API can test how strong an unobserved confounder would need to be to overturn our conclusions. For example, DoWhy can add a fake confounder to see if it breaks the effect. Documenting such checks (even qualitatively) shows we tried to stress-test the result.
- **Confidence Intervals:** Use the built-in bootstrapping in DoubleML/EconML to get confidence intervals for $\hat{\theta}$. If the interval excludes 0, we have evidence the effect is real.

If any check fails (e.g. model is too unstable, or effect flips sign), we must revisit the data and model (perhaps add more controls, or try a different model). We will **report all checks** and their outcomes candidly.

Discount Allocation Optimization

Once we have an ITE estimate for each customer (estimated *increase in expected purchase value due to giving them a discount*), we want to maximize total gain under the budget. A simple formulation:

- Let `v_i = ITE_i × price_i` (estimated incremental revenue if customer *i* is targeted).
- Let `c_i` = cost of discount for customer *i* (could be the discount amount or fixed cost).
- Total budget `B`.

We solve: **maximize** Σ v_i * t_i subject to Σ c_i * t_i $\leq$ B, where t_i $\in$ {0,1} indicates targeting customer *i*. This is a 0-1 knapsack (or integer linear) problem. In practice, if each discount costs a fixed percentage of purchase, `c_i = price_i * discount_rate`. We can use Python optimization: e.g. `scipy.optimize.linprog` (if we relax integrality) or a small integer solver (like `pulp` or SciPy's new `milp` interface).

Example code (using linear programming with a relaxation):

```
import numpy as np
from scipy.optimize import linprog

v = np.array(v_i_list)      # incremental values
c = np.array(c_i_list)      # costs
n = len(v)
# We want to maximize v^T * t, so minimize -v^T * t
# Constraint: c^T * t <= B, and 0 <= t_i <= 1
A = [c]
b = [B]
res = linprog(c=-v, A_ub=A, b_ub=b, bounds=[(0,1)]*n, method='highs')
t_opt = res.x  # fractional solution; we might need to round or add integer
constraints
```

For an integer solution (each customer either gets or doesn't get the discount), we'd use an integer solver. For small groups, rounding the top fractions also often works (e.g. target customers with highest *v_i/c_i* ratio first until budget runs out). The goal is to **maximize estimated total uplifted revenue**.

After solving, we'll review the solution: for example, ensure it favors high-ITE customers (sounds obvious, but if not, something is wrong). We will also check the optimization results with an example: e.g. suppose budget is enough for 100 discounts, does it pick the top-100 ITE customers?

Evaluation Metrics & Visualization

To assess performance, we use metrics from *uplift modeling* and causal inference:

- **Uplift / Qini Curve:** Plot the *uplift curve* (also called a Qini curve)【53†L65-L74】. This shows, for the top φ fraction of customers (ranked by predicted ITE), the gain in response compared to random. In

practice, we sort customers by estimated ITE, then plot cumulative incremental sales versus φ. The area between our curve and the diagonal line (random targeting) is the Qini coefficient【53†L65-L74】. A larger area means better targeting. (See pylift [53] for formal definition.)

- **Gain Chart:** Alternatively, a *cumulative gain chart* shows cumulative incremental revenue on the y-axis vs. number of customers targeted. If our DML estimate is good, the early part of the curve should rise quickly (we capture large lift by targeting few customers).
- **Table of Metrics:** Compute numeric metrics like mean ITE, mean ITE for top decile, and the Qini coefficient. Also report the estimated ROI: e.g. total uplifted revenue divided by total discount cost.
- **Example Charts:**
- *Histogram of ITE estimates* (to see the distribution).
- *Bar chart of top-10 customers by ITE.*
- *Flowchart:* we will include a pipeline flowchart (mermaid) to illustrate the process.

Mermaid Pipeline: Below is an example flowchart of our pipeline (use the `mermaid` code block in markdown):

```
flowchart LR
    A[Raw Data] --> B[Preprocessing & Feat. Eng]
    B --> C[First-Stage Models<br/>(Predict Y and D)]
    C --> D[Double ML Estimation<br/>(Compute ITE)]
    D --> E[Validation<br/>(Placebo & Balance Checks)]
    D --> F[Uplift Metrics<br/>(Qini, etc.)]
    D --> G[Optimization]
    G --> H[Discount Plan & ROI]
    F --> H
    H --> I[Results & Reporting]
```

(Readers should imagine each box as a step: clean data → fit models → compute ITE → check validity → optimize discounts → produce report.)

Chart Recommendations: As part of the final presentation, we recommend:

- A **Qini/Uplift curve** chart (cumulative incremental response vs. % of customers targeted). This visualizes targeting benefit.
- A **bar chart or histogram of ITEs**, highlighting mean uplift. For example, “Average ITE = \$X per customer; top decile customers have Y% higher uplift” to give intuition.
- A **budget allocation summary**: e.g. bar or pie chart showing how budget is used across segments or top customers.

These visuals (tables plus charts) will make the results concrete. All charts should have clear labels (e.g. “Customers targeted (%)” on x-axis, “Incremental Sales” on y-axis).

Hardware/Software Environment

This project can be done on a typical data science workstation. **Software/libraries:**

- **Python 3.8+**. Key packages: `numpy, pandas, scikit-learn` (general ML), `econml` or `DoubleML` (causal DML), `matplotlib/seaborn/plotly` (visualization), `scipy` (optimization). Optionally, `DoWhy` for additional refutations, `pylift` or `scikit-uplift` for evaluation. Use `pip` or `conda` to manage dependencies.
- **Jupyter notebooks** or similar for development and analysis. Keep code modular.
- **Compute needs:** For a moderately sized dataset (thousands to tens of thousands of customers), no special hardware is needed – a laptop or server with ~8–16 GB RAM and a modern CPU is sufficient. If data were huge or models complex, we might use a GPU (e.g. for XGBoost) or cloud VM, but for our project example this is likely unnecessary.
- **Reproducibility:** Store all code in a Git repo. Use a `requirements.txt` or `environment.yml`. Consider a Dockerfile for the final environment if time allows.

Challenges & Mitigations

- **Data Quality:** Real marketing data can be messy or incomplete. If key features are missing (e.g. competitor prices), our estimates may be biased. We must document any missingness or proxies used. Mitigation: use synthetic data to test extremes; be upfront about assumptions.
- **High Variance in ITE:** Individual treatment effects can be noisy. The DML method reduces bias but may have high variance, especially with limited data. We can mitigate by cross-fitting, bagging, or ensembling multiple causal models. Providing confidence intervals helps convey uncertainty.
- **Confounders/Omitted Variables:** If we've omitted an important confounder, the effect estimate is not reliable. This violates the "no unobserved confounders" assumption. We mitigate by careful feature selection (domain knowledge) and discussing this openly. We might use proxy variables as controls.
- **Hyperparameter Tuning Complexity:** Tuning two ML models (for Y and D) plus the final model can be intensive. We should start with default reasonable models (e.g. medium-depth forests) and only tune the most important hyperparameters (like tree depth) to avoid overfitting time.
- **Optimization Reality:** The knapsack model assumes the ITEs and costs are known exactly and independent. In reality, giving a discount may slightly alter future behavior (dynamic effect). We assume short-term horizon. We'll discuss these limitations.
- **Ethical Considerations:** Targeting discounts could lead to perceived unfairness. We should ensure we are not discriminating (e.g. by sensitive attributes). We must mention any checks for bias (e.g. compare ITEs across demographics) and ensure compliance with data privacy rules.

In all, we'll **flag potential issues** as we go (e.g., "If budget is too small, random noise may dominate our allocation decisions") and outline how to detect and address them.

Deliverables Checklist

- **README:** Explains how to run the code and replicate results.
- **Final Report:** This document.
- **Code:** Jupyter notebooks or scripts implementing the pipeline, with comments and citations.

- **Presentation Slides:** Summarizing objectives, methods, key results (tables/charts), and business recommendations.
- **Demo:** e.g. a short video or interactive demo showing the ITE results and optimized plan. Even a well-commented Jupyter notebook can serve as a demo by walking through results.
- **Style:** Use clear English, short paragraphs, and include cited sources for any factual claims (especially on DML theory or dataset descriptions)【21†L11-L19】【42†L43-L49】.

Timeline (Gantt-like)

Below is a simplified Gantt-style schedule summarizing the above milestones:

Weeks	Milestone / Output
1–2	Literature review; data sources finalized; causal DAG and plan settled.
3–4	Data ingestion and cleaning complete; exploratory analysis report.
5–6	Prototype DML models trained; preliminary ITE estimates obtained.
7–8	Model validation done; performance tuned; final ITE estimates.
9–10	Discount optimization implemented; pipeline integration completed.
11	Evaluation metrics computed; charts created; report draft written.
12	Slides ready; final report polished; demo tested; rehearsal done.

Meeting these milestones ensures steady progress. Each week's end, one team member (or pair) should update the status and integrate changes into a shared repo.

Recommended Resources

- **DoubleML documentation** – official guide and examples on DML (Python/R)【21†L11-L19】【56†L332-L334】.
- **EconML docs** – explains DML and other causal estimators (Microsoft's econML library)【25†L14-L21】.
- **Chernozhukov et al. (2018)** – original paper on Double/Debiased ML (The Econometrics Journal)【56†L332-L334】.
- **DoWhy documentation** – for causal inference and refutation methods.
- **Pylift / scikit-uplift** – libraries with examples of uplift modeling and Qini curves (see [53]).
- **Scipy Optimize documentation** – for linear and integer programming solvers.

These sources (and their cited references) provide deeper technical guidance. We will rely mainly on primary documentation and peer-reviewed sources for methodology【21†L11-L19】【25†L14-L21】.

Overall, this project brings together causal inference (via Double ML) and optimization. It will challenge our assumptions and rigor, but by following this plan and validating every step, we will produce a solid, honest analysis that balances ambition with reality.
